CIS-1 · TUTORIAL 2 · BACKGROUND READING

What is an MD engine, and what do NAMD and OpenMM actually do?

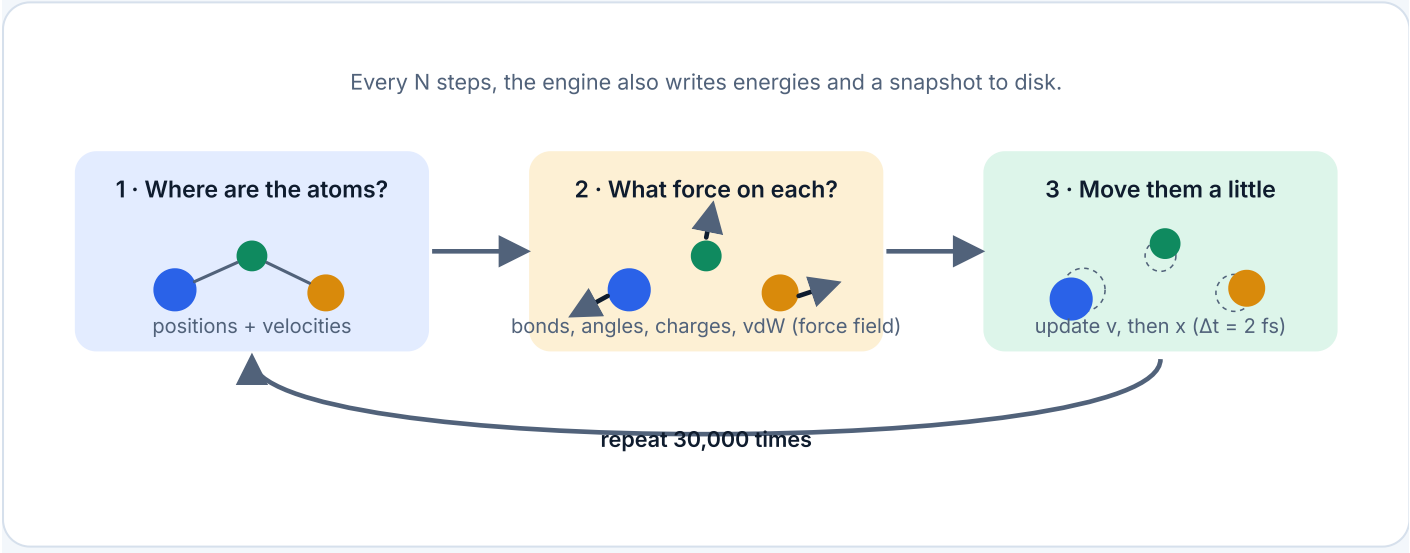
A molecular dynamics (MD) engine is the program that moves atoms. You describe the molecule and the rules. It calculates forces and steps time forward, millions of times, and writes down what happened.

NAMD · config file | OpenMM · Python script | same system: Trp-cage, 6,215 atoms

1 · THE IDEA

Every MD engine runs the same loop

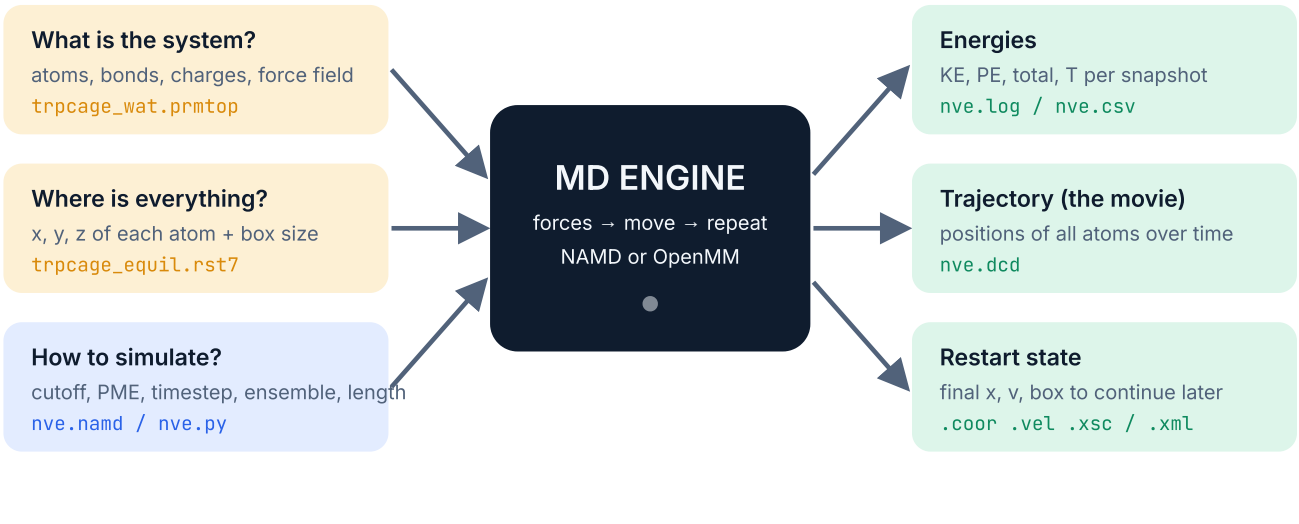
Newton's second law, repeated. One loop turn is one **timestep** (2 femtoseconds in our tutorial). A 60 ps run is 30,000 turns.



2 · THE ENGINE AS A BLACK BOX

What goes in, what comes out

You never edit the loop. You hand the engine three things and collect what it writes.



3 · MEET THE ENGINES

Same job, two different styles

Both implement the same physics. They differ in how you talk to them.

NAMD

Nanoscale Molecular Dynamics · University of Illinois (Theoretical and Computational Biophysics Group)

- A stand-alone **program**. You write a plain-text **config file** and run it from the terminal.
- Built for **big biomolecular systems** on many CPU cores or clusters (viruses, membranes, millions of atoms).
- Pairs with **VMD** for building and viewing systems.
- Native inputs are CHARMM style (PSF + PDB); it also reads Amber files, as we do.

You do: edit `nve.namd`, run `namd2 +p4 nve.namd > nve.log`

OpenMM

A **library** for molecular simulation · Stanford / Pande and Eastman labs, open source

- A **Python library**. You write a short **script** that builds and runs the simulation.
- Built for **speed on GPUs** and for **flexibility**: custom forces and integrators in a few lines.
- Reads Amber, CHARMM and GROMACS inputs; easy to mix with numpy and matplotlib.
- Runs on CPU too, only slower (our 60 ps run: about 30 min on CPU vs 3.5 min in NAMD).

You do: edit `nve.py`, run `python nve.py`

	NAMD	OPENMM
You give it	a config file of keywords	a Python script of function calls
Strength	huge systems, many CPU cores	GPU speed, scripting freedom
Ensemble is set by	which keywords are present (`langevin` , `langevinPiston`)	which integrator and forces you add
Units	Å · fs · kcal/mol	nm · ps · kJ/mol

4 · INPUTS

Three shared files, plus one settings file

The two engines read the **same** topology and coordinates. Only the settings differ, and they describe the same recipe in two languages.

FILE	HOLDS	USED BY
`trpcage_wat.prmtop`	atom types, charges, bonds, angles, dihedrals, LJ parameters (the force field)	both
`trpcage_equil.rst7`	x, y, z of all 6,215 atoms and the box size	both
`nve.namd`	settings as `keyword value` lines	NAMD
`nve.py`	settings as Python code	OpenMM

THE SAME FOUR DECISIONS, WRITTEN BOTH WAYS

NAMD

```
# load force field + coordinates
amber       yes
parmfile    trpcage_wat.prmtop
ambercoor   trpcage_equil.rst7
```

```
# nonbonded interactions
cutoff   9.0
PME      yes
```

```
# 2 fs steps, keep X-H bonds rigid
timestep    2.0
rigidBonds  all
```

```
# NVE = no thermostat, no barostat
# (just do NOT write langevin ...)
temperature 300
run         25000
```

OpenMM

```
prmtop =
app.AmberPrmtopFile('trpcage_wat.prmtop')
inpcrd =
app.AmberInpcrdFile('trpcage_equil.rst7')
```

```
system = prmtop.createSystem(
    nonbondedMethod=app.PME,
    nonbondedCutoff=0.9*unit.nanometer)
```

```
constraints=app.HBonds   # inside
createSystem
integrator =
mm.VerletIntegrator(0.002*unit.picoseconds)
```

```
# NVE = Verlet integrator, no barostat
sim.context.setVelocitiesToTemperature(300*u
nit.kelvin)
sim.step(25000)
```

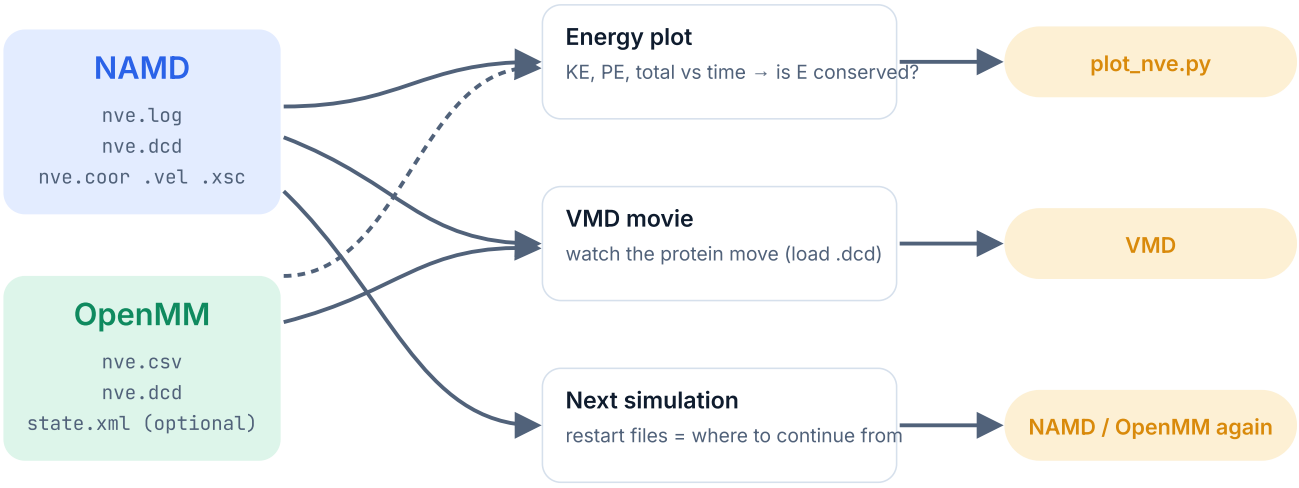
⚠ **Watch the units.** NAMD reports energy in **kcal/mol** and distance in **Å**; OpenMM uses **kJ/mol** and **nm**. When you compare plots, convert first: **1 kcal = 4.184 kJ**.

ENERGY	LENGTH	TIME	KEY
kcal/mol · kJ/mol	Å · nm (1 nm = 10 Å)	fs · ps (1 ps = 1000 fs)	NAMD · OpenMM

5 · OUTPUTS

What you get back, and what to do with it

Each output file has one job. Most are read by another program, not by you.



FILE BY FILE

FILE	ENGINE	WHAT IT IS	OPEN IT WITH
`nve.log`	NAMD	text. Lines starting `ENERGY:` hold energies in kcal/mol; the `ETITLE:` line names the columns	any text tool, `grep`, Python
`nve.csv`	OpenMM	comma-separated table: step, time, PE, KE, total, T (kJ/mol)	numpy, Excel
`nve.dcd`	both	binary trajectory: coordinates of every atom, every few hundred steps	VMD (needs the prmtop too)
`.coor / .vel / .xsc`	NAMD	binary final positions, velocities, box. Used to restart	NAMD only
`state.xml`	OpenMM	saved state (positions, velocities, box) via `sim.saveState()`	OpenMM only

6 · REMEMBER THIS

Four things to take away

An engine is a loop.

Forces, then move, then repeat. Everything else is bookkeeping.

Inputs = system + coordinates + settings.

Our two engines share the first two.

NAMD = config file. OpenMM = Python script.

Different syntax, same physics.

Outputs answer questions.

Log or CSV for energies, DCD for the movie, restart files to continue.

System: Trp-cage (PDB 1L2Y), Amber ff14SB, TIP3P water, 6,215 atoms. NAMD 2.14, OpenMM 8.5.